

Hi-Tec Shopify Automation

Deep Dive

Explainer generated for portfolio review

Covers the code as it exists in the repository, 2023 client work by Salman Adnan

Contents

1	What this project is	3
1.1	The deliberate constraint: no Shopify API	3
2	Repository layout	4
3	Core library used everywhere: pandas	4
4	Tool 1: quantity_pricing_updater	4
4.1	Purpose	4
4.2	Invocation	5
4.3	The matching problem	5
4.4	Algorithm	5
4.5	Why filter by brand first	6
4.6	Traced example from the sample data	6
5	Tool 2: dimensions_extractor	6
5.1	Purpose	6
5.2	Invocation	6
5.3	How the extraction works	6
5.4	The label-then-value assumption, and the “width” fallback	7
5.5	A note on a debug artifact	7
6	Tool 3: stock_updater_gui	8
6.1	Purpose	8
6.2	Window layout	8
6.3	Feature 1 and 2: the same logic as the CLI tools, generalized to XLSX	8
6.4	Feature 3: variant detection	9
6.5	Non-destructive saving: <code>save_data</code>	10
7	Packaging: PyInstaller	10
8	Pandas correctness pitfalls actually present in this code	10
9	Sample data and what was verified	11

10 Lessons and acknowledged limitations	11
11 Glossary	11

1 What this project is

Hi-Tec Bearings is a retailer that sells bearings (mechanical parts that let one part of a machine rotate smoothly against another) through an online store built on **Shopify**, a hosted e-commerce platform. Shopify lets a store owner export its entire product catalogue as a spreadsheet file (**CSV** — comma-separated values, a plain-text table format — or **XLSX**, the Excel spreadsheet format), edit it, and re-import it to update the store in bulk.

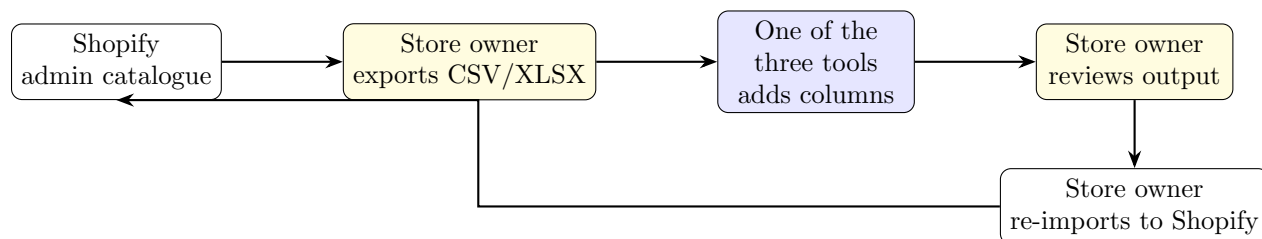
Hi-Tec kept its warehouse stock records (how many of each bearing they physically have, and what it costs them) in a separate spreadsheet from the Shopify catalogue. Keeping the two in sync by hand, one row at a time, took hours every update cycle. This project is three small Python programs, all written by the same author, that automate that reconciliation:

1. **quantity_pricing_updater** — copies live warehouse quantity and cost figures into a Shopify export.
2. **dimensions_extractor** — pulls bearing measurements (bore, outside diameter, width) out of HTML product-description text and turns them into real spreadsheet columns.
3. **stock_updater_gui** — a desktop application (no command line needed) that bundles both of the above plus a third feature, automatic variant labelling, behind three buttons, for handing to a non-technical client.

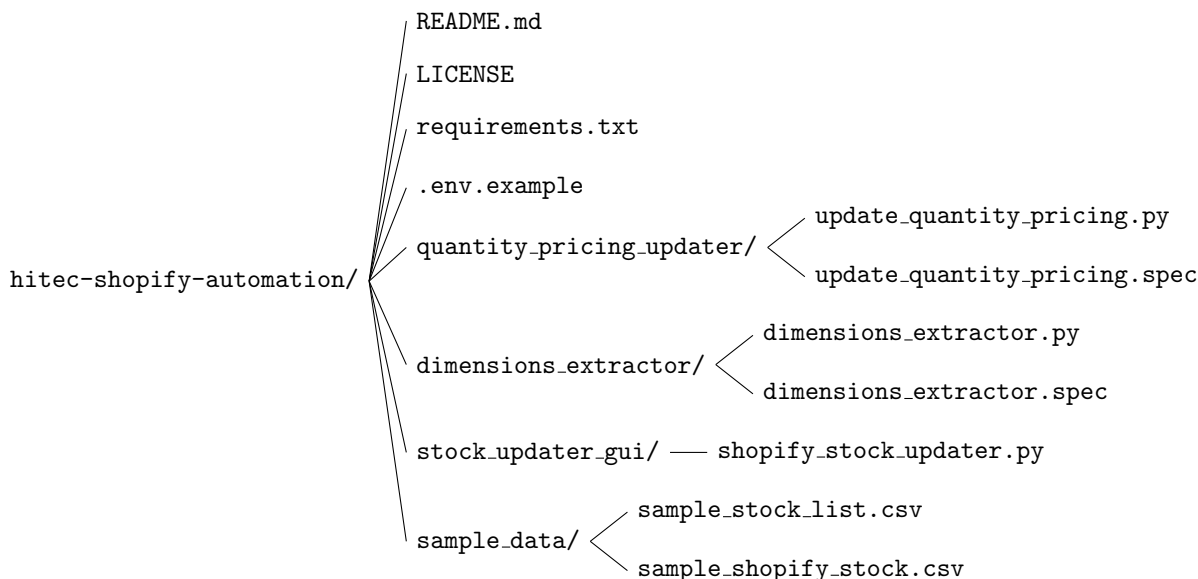
1.1 The deliberate constraint: no Shopify API

A **web API** (Application Programming Interface) would normally let one program talk directly to another system — here, that would mean the script could read and write Shopify’s product catalogue over the internet automatically. This project does not do that. Every tool reads a spreadsheet file that a human has already exported from the Shopify admin panel, adds columns to it, and writes a new spreadsheet file that the same human reviews before re-importing it into Shopify by hand.

This was a deliberate choice (documented by the author in the project’s README), not an oversight: it keeps a human reviewing every catalogue change before it goes live, and it avoids the complexity of authenticating against a live store. The trade-off is that the tools cannot rely on Shopify’s internal, guaranteed-unique product IDs to match records — only on text fields (titles, variant option values) that happen to appear in the exported file, which is a much less reliable key.



2 Repository layout



- `README.md` — usage, design rationale, challenges, and lessons learned.
- `LICENSE` — MIT license.
- `requirements.txt` — pinned dependencies (pandas, openpyxl).
- `.env.example` — documents that no secrets or credentials are needed.
- `*.spec` files — PyInstaller build configs for the two CLI tools.
- `sample_data/` — fabricated stand-in for the client’s real data (see Section 6 onward for context).

There is no automated test suite and no CI configuration in this repository; this is called out explicitly in the README’s “what I’d do differently” section, discussed in Section 10 of this document.

3 Core library used everywhere: pandas

All three tools depend on **pandas**, a Python library for working with tabular data. Its central object is the **DataFrame**: an in-memory table with named columns, similar to a spreadsheet, that supports filtering rows, joining tables, and reading/writing CSV or Excel files with one function call each (`pandas.read_csv`, `pandas.read_excel`, `.to_csv`, `.to_excel`). Every tool in this project follows the same shape: read a DataFrame in, transform it, write a DataFrame out.

`requirements.txt` pins `pandas>=2.0,<3.0`. The reason is explained in Section 8: the code writes into DataFrames in a way that depends on pandas 2.x’s permissive (if warning-prone) behaviour, and would silently misbehave under pandas 3.0’s stricter copy-on-write model.

4 Tool 1: `quantity_pricing_updater`

4.1 Purpose

`update_quantity_pricing.py` takes two spreadsheets — the internal warehouse **stock list** and a Shopify product export — and adds two new columns to the Shopify export: **Updated Quantity**

and **Updated Cost**, populated with the current warehouse figures for each matching product. The result is saved next to the Shopify input file as `updated_<original filename>.csv`, so the original export is never modified in place.

4.2 Invocation

```

1 # Headless (paths given on the command line):
2 python quantity_pricing_updater/update_quantity_pricing.py \
3     sample_data/sample_stock_list.csv sample_data/sample_shopify_stock.csv
4
5 # Interactive (no arguments): a file-picker dialog opens for each input,
6 # using tkinter's askopenfilename, and keeps re-opening until a file is chosen.
7 python quantity_pricing_updater/update_quantity_pricing.py

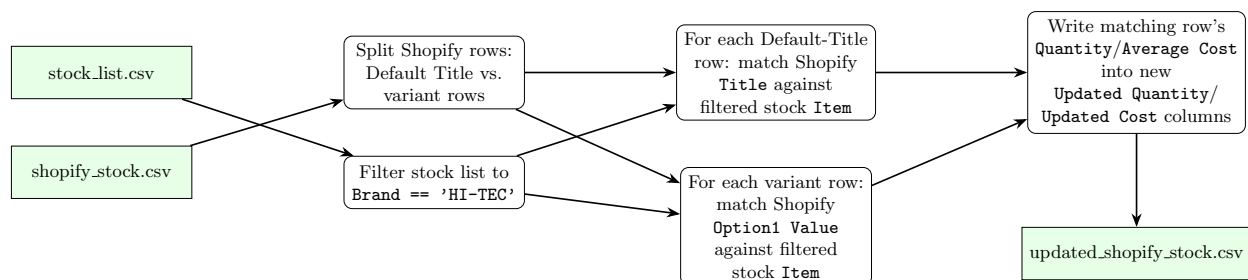
```

`tkinter` is Python's built-in library for small graphical interfaces; `askopenfilename` is the one function from it used here, purely to pop open a native “choose a file” dialog box.

4.3 The matching problem

The two spreadsheets do not share a common ID column. The stock list is keyed by an **Item code** (e.g. 6205) and a **Brand**; the Shopify export is keyed by **Title** for products with only one variant, and by **Option1 Value** for products that have variants (e.g. different bearing suffixes sold under one product page). A further complication: the same item code can appear more than once in the stock list under different brands — Hi-Tec resells other brands' bearings too, and only the HI-TEC-brand row is the one whose quantity and cost should flow into the Shopify export. The sample data reproduces this: 6205 and 6304 2RS each appear twice in `sample_stock_list.csv`, once as HI-TEC and once as OTHER.

4.4 Algorithm



In prose, the script:

1. Reads both CSVs into pandas DataFrames.
2. Splits the Shopify DataFrame into two views: rows where `Option1 Value == "Default Title"` (single-variant products, matched by product **Title**) and rows where it is anything else (multi-variant products, matched by the variant's **Option1 Value**, e.g. "22210 K").
3. Initializes **Updated Quantity** and **Updated Cost** to 0 on the default-title view.
4. For each row in the relevant Shopify view, checks whether its key (**Title** or **Option1 Value**) appears in the stock list *after* filtering the stock list to `Brand == 'HI-TEC'`. If it does, it looks up that item's **Quantity** and **Average Cost** in the filtered stock list and writes both

values into the two new columns for that row, using pandas' `.loc[]` label-based indexing (a way of selecting rows/columns by name rather than position).

5. Recombines the two views with `pandas.concat` and writes the combined result to `updated_<name>.csv`.

4.5 Why filter by brand first

Without the `Brand == 'HI-TEC'` filter, `stock_list[stock_list['Item'] == x]` could return more than one row for the same item code, and pandas would either raise an error or silently pick an arbitrary one — in this data, that could just as easily be the `OTHER` brand's stale quantity and wrong cost. Filtering by brand before matching removes the ambiguity entirely; the sample data's two duplicated item codes (6205, 6304 2RS) exist specifically to demonstrate that the tool picks the HI-TEC row correctly in both cases.

4.6 Traced example from the sample data

Using `sample_stock_list.csv` and `sample_shopify_stock.csv`: Shopify's 6205 product (`Option1 Value = "Default Title"`) has its Title "6205" matched against the stock list filtered to HI-TEC, which contains 6205 at Quantity 55, Average Cost 50 (row 1 of the stock list) — not the competing `OTHER` row (Quantity 3, cost 45). The variant product 22210 has two Shopify variants, "22210 K" and "22210 E"; each is matched independently against the corresponding `Item` value in the stock list (15/172 and 10/185 respectively).

5 Tool 2: dimensions_extractor

5.1 Purpose

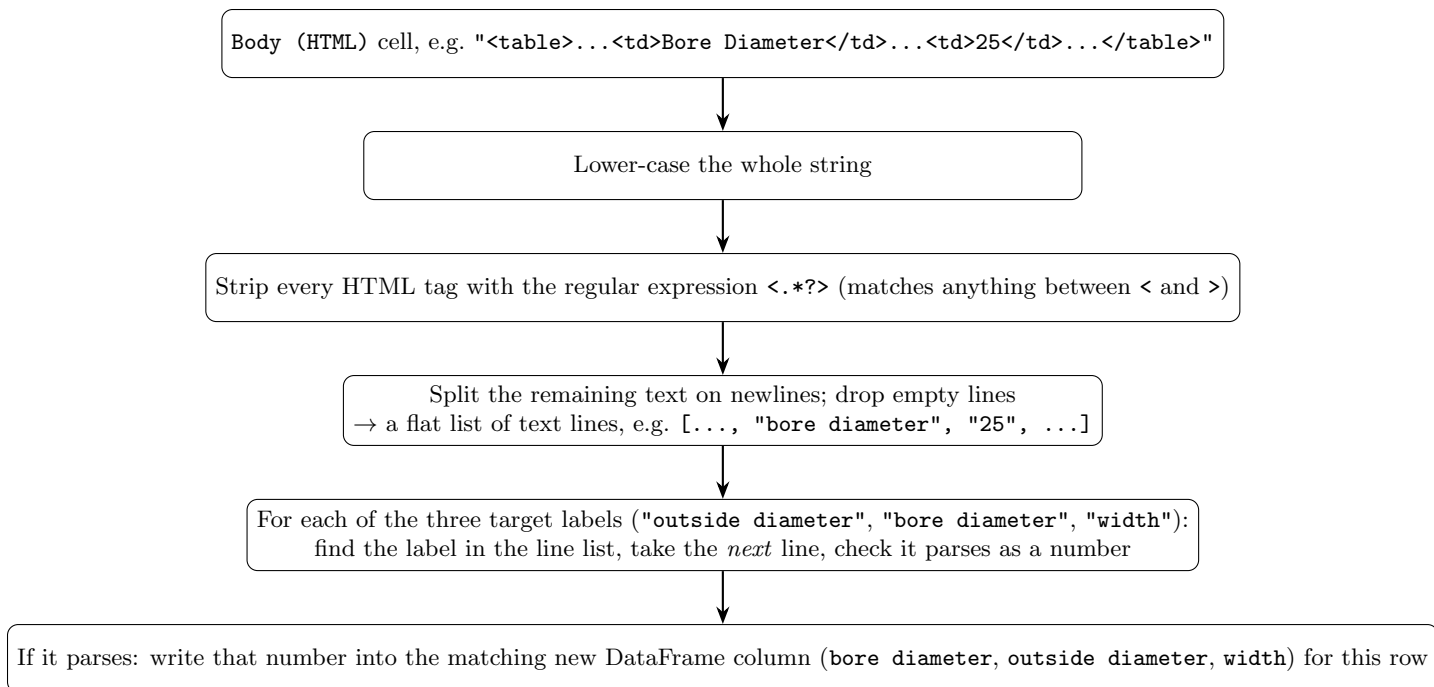
Shopify stores each product's description as one blob of HTML (the markup language used to structure web pages — tags like `<table>`, `<tr>`, `<td>` describe a table's rows and cells). Hi-Tec's product descriptions embed a small dimensions table inside that HTML: bore diameter, outside diameter, and width, in millimetres. Those numbers are not available as normal spreadsheet columns, so they cannot be used as **filterable metafields** (Shopify's mechanism for storefront filters like "show me all bearings with a 25 mm bore"). This tool reads the HTML, extracts the three numbers, and writes them into three new real columns.

5.2 Invocation

```
1 python dimensions_extractor/dimensions_extractor.py sample_data/sample_shopify_stock.csv
```

As with the first tool, omitting the argument opens a file-picker dialog instead.

5.3 How the extraction works



A **regular expression** (or **regex**) is a pattern language for matching text. The pattern used here, `<.*?>`, means: an opening angle bracket, then any characters (`.*`), matched as few as possible at a time (`?`, making the match **non-greedy** so it stops at the *first* closing bracket rather than the last one in the whole string), then a closing angle bracket. Applied repeatedly, this deletes every HTML tag and leaves only the human-readable text.

5.4 The label-then-value assumption, and the “width” fallback

The extractor assumes each dimension appears as a line containing only the label (e.g. the text line `"bore diameter"`), immediately followed by a line containing only the number. This matches how the sample HTML’s table renders once tags are stripped: a cell with just the label text, then a cell with just the number, each becoming its own line. `outside diameter` and `bore diameter` are searched for as either an exact line match or (in the code path for `outside diameter` and `bore diameter` specifically) as a substring within any line. `width` is checked for an exact line match first, and if not found, the code falls back to looking for the label `"total width"` instead, because some of the client’s product descriptions use that longer label instead of plain `"width"`.

This is explicitly a template-shaped heuristic, not a general HTML table parser: the README states plainly that it depends on the exact label-then-value line ordering produced by this store’s product-description template, and would silently miss dimensions on any description written differently. A parser that read the actual `<table>/<tr>/<td>` structure (rather than flattening everything to a line list first) would not have this fragility, at the cost of more code.

5.5 A note on a debug artifact

The loop prints `overall_dimensions` and a row of dashes for every product row, and, in the “width” branch, an extra diagnostic line (`print('check width', 'total width' in text)`). These are development-time debug statements left in the shipped script; the README’s “what I’d do differently” section flags this directly and recommends removing them or replacing them with proper

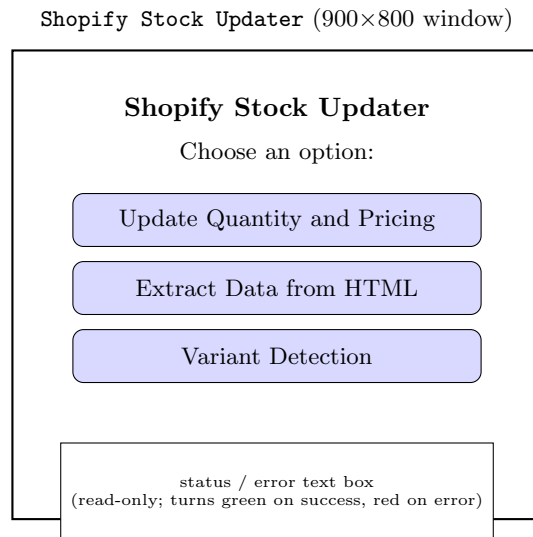
logging (a more structured, filterable way of recording what a program is doing, typically via Python’s `logging` module instead of bare `print`).

6 Tool 3: `stock_updater_gui`

6.1 Purpose

`shopify_stock_updater.py` is the version actually handed to the client. It is a **desktop GUI application** (Graphical User Interface — a program controlled with windows, buttons, and dialogs rather than a command line) built with `tkinter`, Python’s built-in GUI toolkit. It wraps the logic of the two tools above, plus a third capability (variant detection), behind three buttons, so a non-programmer can run the whole workflow by double-clicking an executable and clicking a button.

6.2 Window layout



Each button is wired, via `tkinter`’s `command=` argument, to one Python function: `update_quantity_and_pricing`, `extract_data_from_html`, and `variants_detection`. Two small handlers, `on_enter/on_leave`, change a button’s background colour on mouse hover (bound to `tkinter`’s `<Enter>/<Leave>` pointer events) as a visual affordance.

6.3 Feature 1 and 2: the same logic as the CLI tools, generalized to XLSX

`update_quantity_and_pricing` and `extract_data_from_html` follow the same matching and extraction logic already described in Sections 4 and 5, with three differences suited to a non-technical client:

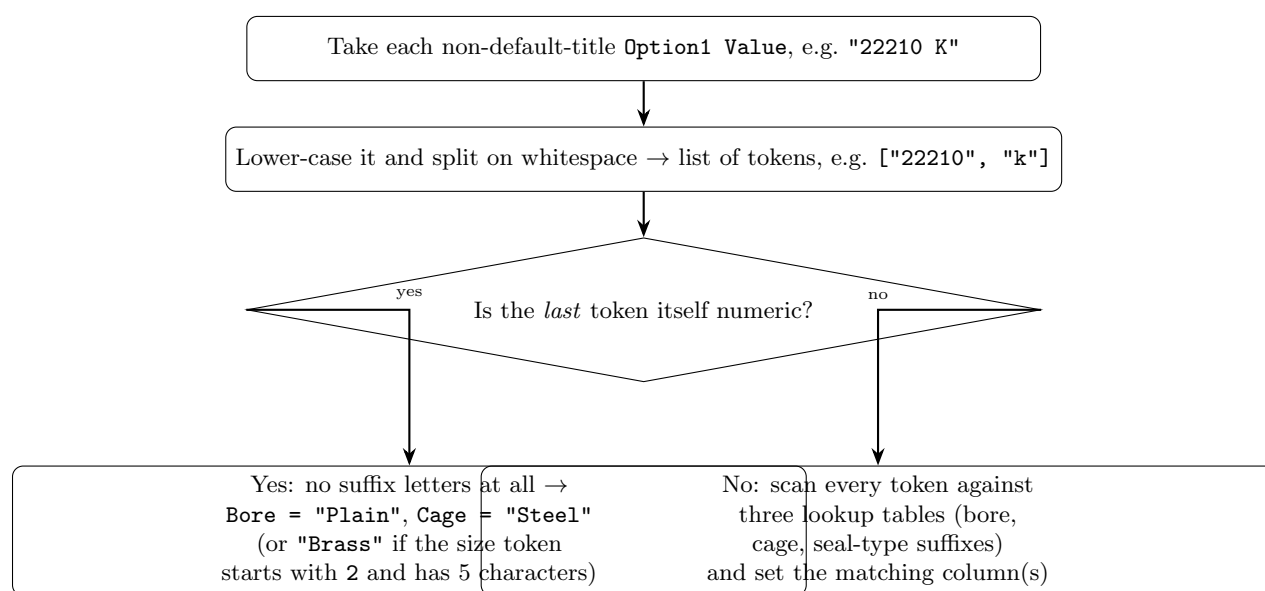
- `open_file` accepts either `.xlsx` or `.csv` and returns a flag saying which; the caller then calls `pandas.read_excel` or `pandas.read_csv` accordingly. Output is always written as `.xlsx` via `pandas`’s `.to_excel` (which depends on the `openpyxl` library declared in `requirements.txt` to actually write the `.xlsx` file format).
- Before matching, `stock_list['Item']` is passed through `convert_int_float`, coercing item codes to numbers where possible. This matters because Excel commonly stores a code like

6205 as a genuine number rather than as text, so the same value needs to compare equal whether it came from an Excel numeric cell or a CSV text field.

- Output filenames are prefixed `updated_p_` (pricing) or `updated_h_` (HTML/dimensions) instead of just `updated_`, and saving goes through the shared `save_data` helper described next.

6.4 Feature 3: variant detection

Bearing product names encode extra specification detail as suffixes after the base size, e.g. "22210 K" or "6304 2RS". `variants_detection` parses these suffixes into three new columns: **Bore**, **Cage** (the cage is the internal structure that holds a bearing's rolling elements apart; different materials — steel, brass, fibre, polyamide — suit different conditions), and **Seal Type** (whether and how the bearing is sealed against dust and moisture).



The three lookup tables in the code are parallel lists (a suffix token list and a matching human-readable name list at the same index):

Category	Suffix tokens checked	Mapped to
Bore	k	Tapered
Cage	e, f, m, ecp, ca, mb, cc	Steel, Fibre, Brass, Polyamide, Brass, Steel, Brass
Seal Type	2rs, rs, z, 2z	2RS, rs, z, 2z

After the main token scan, there is a second pass specific to the cage suffixes `e/ca/mb` and `cc`: if the numeric size token (the one found not to be purely a number-check target, tracked via the loop variable that holds the index of the last numeric token seen) starts with the digit 2 and is five characters long, the cage is overridden to **Brass** (for `e/ca/mb`) or **Steel** (for `cc`). This encodes a bearing-industry convention where a leading 2 in a five-digit size code changes the implied cage material; the exact numbering convention is Hi-Tec's/the bearing industry's own classification and is reproduced here only as implemented in code, not independently verified against a bearings reference.

6.5 Non-destructive saving: `save_data`

All three GUI features funnel their output through one shared function, `save_data`, which never overwrites an existing file. It tries to read the target path with `pandas.read_excel`; if that succeeds (the file exists), it increments a counter and retries with the path rewritten to `(N)originalname.xlsx`, repeating until it lands on a path that does not yet exist. When it finally hits a `FileNotFoundError` (the path is free), it writes the `DataFrame` there and reports the saved path and a success message in the on-screen status box.

7 Packaging: PyInstaller

Both CLI tools (`dimensions_extractor.py` and `update_quantity_pricing.py`) ship with a `.spec` file — a build recipe for **PyInstaller**, a tool that bundles a Python script and everything it needs (the interpreter, installed libraries) into one standalone executable, so an end user can run it without installing Python themselves. Both `.spec` files are close to PyInstaller’s own generated default: a single **Analysis** block naming the entry script, no extra data files or hidden imports declared, producing a console-mode (`console=True`) Windows executable named after the script. The GUI script has no `.spec` file in this repository; the README states the client-facing app was the GUI, implying it was very likely packaged the same way in practice, but no corresponding `.spec` file is present to confirm the exact build settings used for it — this is flagged here as an inferred, not verified, detail.

8 Pandas correctness pitfalls actually present in this code

Two related pandas anti-patterns appear in this codebase, both acknowledged in the README’s retrospective sections:

1. **Chained assignment into a filtered slice.** Code such as `default_title_shopify_stock["Updated Quantity"] = 0` writes into a `DataFrame` that was produced by boolean filtering (selecting the rows where `Option1 Value` equals "Default Title"). pandas cannot always tell whether such a filtered result is an independent copy of the data or a view into the original table, and raises a `SettingWithCopyWarning` to flag the ambiguity. In this code the writes happen to land correctly under pandas 2.x’s default behaviour, but the warning is a sign the pattern is fragile.
2. **Pinning pandas<3.** pandas 3.0 turns on **copy-on-write** by default, a stricter memory model in which every filtered slice is treated as an independent copy unless explicitly told otherwise. Under that model, the same chained-assignment pattern would silently write to a copy that is then discarded, rather than to the data that eventually gets saved — turning a visible warning today into silently lost updates on a pandas 3.0 install. `requirements.txt`’s `pandas>=2.0,<3.0` pin is a direct consequence of this, stated explicitly in the README.

The README’s own assessment (not this document’s added judgement) is that a proper **merge** of the two tables on **Item/Brand** would sidestep both problems, be more correct, and also replace the current per-row Python loops (an $O(n \times m)$ approach — for every Shopify row, the entire filtered stock list is converted to a list and scanned again) with a single vectorized operation, which matters at the original client’s real scale (roughly 1,600 stock rows against an 11,000-line Shopify export).

9 Sample data and what was verified

The original repository contained the client’s real data: about 1,600 warehouse stock rows with real cost and sale prices and supplier names, and an 11,000-line Shopify export including per-item costs. That data is client-confidential and was removed for this portfolio copy. `sample_data/sample_stock_list.csv` and `sample_data/sample_shopify_stock.csv` are fabricated replacements with the same column layout and the same HTML dimensions-table structure, small enough to read in full (reproduced in part in Sections 4.6 and 5 of this document).

Per the README, both CLI tools were re-verified against this sample data: the quantity/cost merge correctly resolves the two duplicated item codes to their **HI-TEC** row, and the dimensions extractor recovers all three measurements from the sample HTML. The GUI was smoke-tested only to the point of confirming its window opens; a full end-to-end run through a live Shopify store (real export, processing, re-import) was performed during the original 2023 engagement and was not repeated for this portfolio version. This document repeats that scope honestly rather than implying a fuller verification took place.

10 Lessons and acknowledged limitations

These are the author’s own retrospective notes from the README, reproduced here because they are part of understanding the project honestly, not independently added critique:

- Matching records with no shared key is mostly about choosing the right disambiguator first — filtering to the **HI-TEC** brand before joining eliminated a whole class of wrong matches no amount of fuzzy title matching would have fixed.
- Flattening HTML to a line list and reading label/value pairs is quick to write but brittle: it depends entirely on the store’s template, and a parser keyed on actual table structure would survive layout changes this approach misses.
- The GUI script duplicates the two CLI scripts’ logic almost line for line; the processing logic should live in one shared module imported by both the CLI and tkinter front ends.
- The matching loops are $O(n \times m)$ and would be a single `merge` call in a corrected version.
- Debug `print` statements were left in from development.
- The dimensions extractor’s label-then-value-line assumption is store-template specific.
- There are no automated tests; even a handful of fixture-based tests around matching and extraction would have caught regressions between the CLI and GUI versions.

11 Glossary

API Application Programming Interface — a defined way for one program to talk to another automatically, without a human in between.

CSV Comma-Separated Values — a plain-text spreadsheet format, one row per line, columns separated by commas.

XLSX The file format used by Microsoft Excel spreadsheets.

DataFrame pandas' in-memory table object: rows and named columns, like a spreadsheet held in a program's memory.

Regular expression (regex) A pattern language for matching or extracting text, e.g. `<.*?>` to find and remove HTML tags.

HTML HyperText Markup Language — the tag-based language used to structure web page content, including Shopify's stored product descriptions.

tkinter Python's built-in library for building simple desktop GUI windows, buttons, and dialogs.

PyInstaller A tool that bundles a Python script plus its dependencies into one standalone executable file for an end user without Python installed.

SettingWithCopyWarning A pandas warning raised when writing into a DataFrame that might be a filtered view rather than an independent copy of the data.

Copy-on-write A memory model (default in pandas 3.0) where a filtered slice of a table is always treated as its own independent copy until told otherwise.

Metafield Shopify's mechanism for attaching extra, filterable structured data (like a numeric bore diameter) to a product beyond its built-in fields.